

GUÍA DE INSTRUCTORES

Misión RoboTech-1 — Shell y Git para principiantes

Versión con `git clone`

Esta versión está pensada para alumnado que **no ha utilizado nunca la Shell ni Git**. El taller comienza con `git clone`: los alumnos descargan un proyecto ya existente y después trabajan sobre él. **No deben ejecutar `git init` después de clonar**, porque un repositorio clonado ya está inicializado.

Idea metodológica

1. Explicar brevemente qué es una terminal, un comando, un argumento y un directorio.
2. Mostrar la URL del repositorio y explicar que el proyecto está almacenado remotamente.
3. Dejar que busquen en la cheatsheet cómo descargarlo.
4. Hacerles observar la carpeta oculta `.git`, sin profundizar en su estructura.
5. A partir de ahí, trabajar siempre dentro del repositorio clonado.
6. Evitar ramas, conflictos, `push` y autenticación durante la actividad principal.

Preparación previa del instructor

Preparar un repositorio remoto accesible por los alumnos con, al menos:

- `README.md`
- `telemetry.log`

Ejemplo de `telemetry.log`:

```
[10:42:01] TEMP 31 OK
[10:42:02] BATT 98 OK
[10:42:03] MOTOR 2210 OK
[10:42:04] TEMP 32 OK
[10:42:05] BATT 91 OK
[10:42:06] TEMP 57 WARNING
[10:42:07] MOTOR 2420 OK
[10:42:08] BATT 12 WARNING
[10:42:09] TEMP 39 OK
[10:42:10] BATT 83 OK
```

El repositorio debe poder clonarse sin que los alumnos tengan que resolver permisos especiales. Para una primera sesión es recomendable un repositorio público o un mecanismo de acceso ya probado.

Misión 0 — Recuperar el software

Solución orientativa

```
git clone URL_DEL_REPOSITORIO
cd NOMBRE_DEL_REPOSITORIO
ls
ls -a
git status
```

Al ejecutar `ls -a` deberían poder observar la carpeta oculta `.git`.

Cómo ayudar

Explicación breve recomendada:

“`git clone` descarga un proyecto que ya existe. Como también descarga la información de Git, no necesitamos hacer `git init`.”

Misión 1 — Solución

Solución orientativa

```
whoami
who
pwd
date
ls
echo $HOME
echo $SHELL
```

Cómo ayudar

Si preguntan directamente qué escribir, preguntar primero qué información quieren obtener y remitirles a la sección correspondiente de la cheatsheet.

Misión 2 — Solución

Solución orientativa

```
mkdir mision_robotech
cd mision_robotech
mkdir telemetria
mkdir informes
touch checklist.txt
ls
cp checklist.txt checklist_backup.txt
touch prueba.txt
rm prueba.txt
```

Cómo ayudar

Esta misión debe ir despacio. Para alguien que empieza, entender la diferencia entre archivo, directorio, copiar, borrar y cambiar de carpeta ya es un objetivo importante.

Misión 3 — Solución

Suponiendo que el alumno está dentro de `mission_robotech`, puede volver a la raíz del repositorio con:

```
cd ..
```

Después:

Solución orientativa

```
cat telemetria.log
head telemetria.log
tail telemetria.log
grep "TEMP" telemetria.log
grep "WARNING" telemetria.log
grep "WARNING" telemetria.log | wc -l
grep "TEMP" telemetria.log | tail -1
grep "WARNING" telemetria.log > mission_robotech/alertas.txt
cat mission_robotech/alertas.txt
```

Cómo ayudar

No introducir la tubería hasta que entiendan por separado qué hace `grep`, qué hace `wc -l` y qué hace `tail`.
No usar `awk`.

Misión 4 — Solución

Solución orientativa

```
echo $USER
echo $HOME
echo $SHELL

COHETE="RoboTech-1"
DESTINO="Marte"

echo "Preparando $COHETE con destino $DESTINO"

export COHETE
printenv COHETE
```

Cómo ayudar

No hace falta profundizar en procesos padre e hijo. Basta con explicar que una variable exportada puede estar disponible para programas lanzados desde esa Shell.

Misión 5 — Solución

Desde la raíz del repositorio:

Solución orientativa

```
git status
git add .
git status
git commit -m "Trabajo de la mision"
git log
```

Cómo ayudar

La idea que deben llevarse es:

cambioalgo → lopreparo → loguardoenuncommit

No es necesario ejecutar `git push`.

Misión final — Solución orientativa

Una posible solución, partiendo de la raíz del repositorio:

Solución orientativa

```
cd mision_robotech
ls

echo $USER > informe_final.txt
date >> informe_final.txt
echo $COHETE >> informe_final.txt

grep "WARNING" ../telemetria.log | wc -l >> informe_final.txt

cp informe_final.txt informes/

cd ..
git status
git add .
git commit -m "Mision preparada para lanzamiento"
```

No exigir coincidencia literal con esta solución si el resultado es equivalente.

¿Por qué no usar `git init`?

En este taller los alumnos parten de un proyecto ya existente:

repositorio remoto → git clone → repositorio local

Por tanto, después del clone ya existe una carpeta `.git` y el proyecto ya es un repositorio.

`git init` se puede explicar como concepto adicional:

“Si tuviese una carpeta normal y quisiera empezar a controlarla con Git desde cero, usaría `git init`.”

No es necesario que lo ejecuten durante la actividad.

Qué dejamos fuera a propósito

Aunque algunos comandos aparecen en las diapositivas, no son obligatorios en esta primera sesión:

- `git init` como ejercicio obligatorio
- `git push` y `git pull`
- ramas, merges y conflictos
- `diff`
- `sort`
- `chmod`
- `sudo`
- `top` y `htop`
- `tar`
- scripts con bucles
- funciones y flags
- `awk`

Pueden mostrarse como ampliación si algún grupo termina pronto.

Tiempo orientativo

Introducción	10 min
Misión 0: clone	10–15 min
Misión 1	10 min
Misión 2	15–20 min
Misión 3	25–30 min
Misión 4	15 min
Misión 5	15–20 min
Misión final	15–20 min

Tiempo total aproximado: **2 horas**, dependiendo del ritmo del grupo.

Corrección

Para cada misión basta comprobar:

1. ¿Han conseguido el resultado?
2. ¿Saben explicar aproximadamente qué hizo el comando?
3. ¿Han sido capaces de localizarlo en la cheatsheet?

Un error corregido por el propio alumno forma parte del aprendizaje y no debería penalizarse.